Drishti on AWS with Apache Iceberg

The same Drishti on either table format: what is identical, what differs, why, and how to choose between Delta Lake and Iceberg

Status: drishti-iceberg (Iceberg without Hadoop) and the iceberg and iceberg-external connectors built in 2.6.0; mixed Delta and Iceberg storage built in 2.6.7; the AWS scripts in deploy/aws/ are the Delta lake's, their Iceberg variants are proposed
Sources: docs/architecture/DRISHTI_ICEBERG.md, docs/connectors/ICEBERG_CONNECTOR.md, ICEBERG_EXTERNAL_CONNECTOR.md, AWS_DELTA_LAKE.md and the code (Drishti 2.9.4). Companion to Drishti-AWS-Delta-Lake.pptx

Drishti · Any data. Any domain. One grammar.

Copyright © 2026 Ashutosh Sinha. All rights reserved. Proprietary and confidential.

What this deck covers

contents

1 The principle: one Drishti above the storage plugins

2 The picture on AWS, with Iceberg

3 Why S3 and not EFS (the same answer)

4 Iceberg in Drishti, as built

5 Built, designed, proposed

6 Deploying on AWS with Iceberg: catalog, IAM, KMS, network

7 The layout: tables, columns, partitions

8 Files and loading

9 Maintenance

10 Caches and staying fast

11 Delta Lake and Iceberg side by side

12 Which to choose: a guide for a bank

13 Risk workloads on Iceberg: VaR, exposure, official results

14 Migration and coexistence

15 Sizing and cost

16 Runbooks

17 Limits, decisions, related documents

The one rule: the format is a per-connector choice

read this first

Drishti is table-format-agnostic above the storage plugins. Delta Lake or Iceberg is chosen per connector, per table.
Packs, data contracts (JSON Lines in), connector names, searches, BI, the VaR and exposure engines and the UI do not know the format. Either way the lake lives in Amazon S3 and EFS keeps only state and caches.

- **Delta Lake:** `s3://<org>-drishti-<env>-lake/lake/<domain>/<table>/` with `_delta_log/` and `business_date=…/` folders
- **Iceberg:** `s3://…-lake/iceberg/<domain>/<table>/` with `metadata/` (metadata JSON, manifest lists, manifests) and `data/business_date=…/`
- **One switch per table:** a connector file says `plugin: delta` or `plugin: iceberg`; its name (`trade-calypso`) and its kinds stay
- **Never on EFS:** lake data, `_delta_log/`, `metadata/`. EFS holds configuration, users, Drishti*net state and rebuildable caches
- **The honest caveat:** the two plugins do not yet implement the same capabilities: VaR and exposure vectors, official results and the AWS scripts are Delta only today (slide 8, section 13)

Status: what exists today, and how it was tested

read this first

Piece	Status	Where, and how it is tested
drishti-iceberg: Drishti's own Iceberg engine, no Hadoop (read, write, commit, maintain)	**BUILT**	`drishti-iceberg`, `drishti-lakeio`; pyiceberg interop both ways; a class loader that refuses Hadoop
`iceberg` connector (Drishti's layout) and `iceberg-external` (anyone's tables, read-only)	**BUILT**	`plugins/drishti-plugin-iceberg`, `-iceberg-external`; every plugin test on both engines
Iceberg tables in S3: loader writes, connector reads, commits by conditional PUT	**BUILT**	S3Mock (in-process and Docker), MinIO; racing writers on MinIO only; never real AWS S3
REST catalogs, incl. AWS Glue's Iceberg endpoint (SigV4)	**BUILT**	REST protocol tests; SigV4 signature unit test; not run against Glue
Mixed storage: a source system's trades in Iceberg beside Delta (`--store calypso=iceberg`)	**BUILT**	`tools/lake/stores.py`; equality test: 18 answers the same
AWS scripts for an Iceberg lake (load, maintain, lifecycle, alarms, report)	**PROPOSED**	`deploy/aws/delta-lake/` is Delta only; ECS, EFS, IAM, Cloud Map scripts are shared
VaR and exposure inputs (vectors) and official results on Iceberg	**PROPOSED**	the Iceberg plugin has no `vectors()`; publishing writes Delta

01
The principle

One Drishti above the storage plugins: what is identical on Delta and Iceberg, where each plugs in, and how one table switches format

Where Delta and Iceberg plug in, and where they don't

Console and UI · terminals, pages, BI reports, Admin

Server · BI, VaR and exposure engines, captures, Drishti*net (replicas, shards, forwarding)

Packs and data contracts · kinds, promoted columns, JSON Lines in · connector names (trade-calypso)

Engine · SourceRouter: a kind to its connectors, in order; reads, searches, columns, history, scans

Identical on both formats

Nothing above the line knows whether a table is Delta or Iceberg. A pack routes kind `trade` to `trade-calypso`; the file under that name decides the format.

The seam · drishti-api `SourcePlugin`: fetch, days, columnar, columns, scan, dataVersion, heldDates, vectors, documents, tableIdentity …

plugin: delta

- Delta Kernel over `drishti-deltalake` (native) or Hadoop
- tables Drishti or others write
- vectors, whole-day documents

plugin: iceberg

- `drishti-iceberg` (native, default) or Apache Iceberg Java (library)
- Drishti's layout: id, doc, promoted
- loader and maintenance write

plugin: iceberg-external

- `drishti-iceberg` only
- anyone's tables, the row is the document
- read-only

other plugins

- files, S3 objects, PostgreSQL, DuckDB, Cassandra, Aerospike, MongoDB, Redis …

drishti-lakeio (shared): local disk, S3, byte ranges, codecs, footer cache, exclusive create, conditional PUT

Format-specific below the seam only

Storage · local disk or EFS (development, Windows) · Amazon S3 (`s3://`, `s3a://`) · Azure, GCS, HDFS only through the Hadoop/library engines

What is identical, what differs, and why

Layer	Identical on both	Differs	Why it differs
Packs, contracts, connector names	kinds, promoted columns (`layout.<kind>.columns`), JSON Lines input, routes	nothing	the format is below the plugin
Logical table layout	`id`, `doc`, `business_date`, promoted columns; sorted by id; one partition per day; 250,000 rows a file	row groups by rows (Delta, 1,000 rows or 1 MB on AWS) vs by compressed bytes (Iceberg, 1 MB)	Iceberg's writer sizes row groups in bytes
Read paths	id map, one row group per read, column sets, type-ahead, history, BI column scan	Iceberg caches by day fingerprint; Delta by table version	Iceberg's manifests give each day's files cheaply
Where the table's truth is	a folder in S3, Parquet data files	`_delta_log/` commits and checkpoints vs `metadata/` JSON, manifest lists, manifests	the two specifications
Commits on S3	a conditional PUT (`If-None-Match: *`): two writers never both win	Delta: `lake publish` (Python deltalake); Iceberg: the Java loader itself, plus a probe	Drishti's Java Delta writer writes local disk and EFS only
Catalog	none needed (path-based)	Iceberg may also use a REST catalog (Glue, Polaris)	Iceberg's ecosystem is catalog-centred
Maintenance	retention by business date, rewrite drifted days	`lake maintain` + Admin → Lakes (Delta) vs `IcebergMaintenance` CLI	built at different times; no UI for Iceberg yet

The seam in detail: what each plugin implements today

Capability (SourcePlugin method)	delta	iceberg	iceberg-external	Who needs it
one entity, history days, search, reverse links (`fetch`, `days`, `search`, `reverse`)	**BUILT**	**BUILT**	**BUILT** (no `days`)	every page
columns of a day (`columnar`, `columns`), type-ahead index	**BUILT**	**BUILT**	**BUILT** (columns)	searches, pick lists, impact
BI column scan, data version, held dates, row estimate	**BUILT**	**BUILT**	**PROPOSED**	BI store and runs, statements
time travel (*known at*)	**BUILT**	**BUILT**	**BUILT**	every as-known-at view
vector columns (`vectorColumns`, `vectors`): `list<double>` streamed	**BUILT**	**PROPOSED**	**PROPOSED**	VaR, exposure
whole-day documents (`documents`)	**BUILT**	**PROPOSED**	**PROPOSED**	derived pages (faster path)
table identity, holds none (`tableIdentity`, `holdsNone`)	**BUILT**	**PROPOSED**	**PROPOSED**	Drishti*net replicas vs shards
inventory, refresh on signal (`inventory`, `refreshVersions`)	**BUILT**	**PROPOSED**	**PROPOSED**	Admin test, data-load signal

Clean layering, uneven capability. Every Iceberg gap above is an implementation of an existing interface method, not a change to the architecture: the seam is already where it should be.

Switching one table: --store calypso=iceberg


```
python tools/build_demo_lakes.py --store calypso=iceberg        # CALYPSO in Iceberg, ENDUR and MUREX in Delta
python tools/build_demo_lakes.py --store all=iceberg            # every system in Iceberg
```

Before: every system in Delta

trade-endur · `plugin: delta`
data/delta/trading/trade_endur

trade-calypso · `plugin: delta`
data/delta/trading/trade_calypso

trade-murex · `plugin: delta`
data/delta/trading/trade_murex

After: CALYPSO in Iceberg, same connector name

trade-endur · `plugin: delta`
data/delta/trading/trade_endur

trade-calypso · `plugin: iceberg`
data/iceberg/trading/trade_calypso · engine: native

trade-murex · `plugin: delta`
data/delta/trading/trade_murex

What stays: the trading pack's route `trade → trade-endur, trade-calypso, trade-murex`, the kinds, the promoted columns, every page, search, history and BI model.
What changes: one connector file's `plugin`, `engine` and `table`. Proven by `stores_equality.py`: 18 answers the same, all-Delta against mixed.

02

The picture on AWS, with Iceberg

The same flow as the Delta lake, with one step fewer: Drishti's Iceberg loader writes the table in S3 directly

The picture: the whole flow on one page

Sources

- ENDUR, CALYPSO, MUREX, reference
- dumps: JSON / JSON Lines / .gz

S3 · <org>-drishti-<env>-landing

- `<date>/<source>/*.jsonl.gz`
- kept 30 days, then expired (as Delta)

EventBridge Scheduler

- weekday evening load; nightly maintenance
- same schedules as the Delta lake

ECS task · loader · IcebergLoader (engine native), per table, under the table's lock — PROPOSED

- 1 read the landing dumps (JSON Lines: `{domain, kind, id, date, doc, columns}`) 2 external sort per day by id (spill on EFS or ephemeral disk)
- 3 write Parquet + manifests to `s3://…-lake/iceberg/<domain>/<table>/` directly 4 commit: conditional PUT of `vN+1.metadata.json` (or the REST catalog)

S3 · <org>-drishti-<env>-lake (THE LAKE)

- `iceberg/trading/trade_endur/metadata/` v12.metadata.json, snap-….avro, …-m0.avro
- `…/data/business_date=2026-10-20/…parquet` (zstd, sorted by id)
- versioning · SSE-KMS · VPC endpoint only · no S3 expiry of table files

GETs

ECS services drishti-a, -b, -c (server + console) — BUILT

- `plugin: iceberg`, `engine: native`, root `s3://…-lake/iceberg`
- heap: layouts, id maps, column sets, footers, manifests
- Drishti*net between task IPs; seeds by Cloud Map

ECS task · maintenance (nightly) · IcebergMaintenance — PROPOSED

- keep-days, relayout, manifests, expire snapshots

Optional catalog · Glue Iceberg REST, Polaris

- path-based tables need none

The picture, in words: each box

Box	What it is	Key facts	Status
Sources, landing bucket, scheduler	as in the Delta deck	dumps by date and source; 30 days; weekday load and nightly maintenance	**BUILT** (scripts)
Loader task	`IcebergLoader` in the server's exec jar (`-Dloader.main=…IcebergLoader`)	sorts each day by id, writes files of 250,000 rows, commits each day as one overwrite of its partition	**PROPOSED** (ECS)
The lake	Iceberg tables in the lake bucket	`metadata/` + `data/business_date=…/`; files named by absolute S3 paths	**BUILT** (format)
Server nodes	the `iceberg` connector, native engine	refreshes every 10 s (`version-hint.text` or `loadTable` with ETag); reads one row group per entity	**BUILT**
Maintenance task	`IcebergMaintenance`	`--keep-days`, `--relayout`, `--rewrite-manifests`, `--expire-hours`	**PROPOSED** (ECS)
Catalog (optional)	a REST catalog: Glue's Iceberg endpoint, Polaris, Snowflake Open Catalog	commits by the catalog's requirements (409 retried); vended credentials	**BUILT** (client)
EFS	Regional, encrypted, Elastic	state and caches; loader spill and locks; never lake data or `metadata/`	**BUILT** (scripts)

03
Why S3 and not EFS

The Delta deck's answer holds unchanged for Iceberg, with one more reason: Iceberg metadata names every file by its full path

Why S3 and not EFS: the same answer, one more reason

	Amazon S3	Amazon EFS
Cost of 200 TB (list prices)	about **USD 1,400 a month** tiered by business date; USD 4,600 all in Standard	**USD 60,000 a month** in Standard; USD 3,200 in IA plus a charge per byte read
Request rate	≥ 3,500 writes, 5,500 reads a second **per prefix**; each `business_date=` folder is a prefix	one file system's limits, shared by every node, loader and maintenance job
Latency of a read	10–30 ms to first byte; Drishti reads one row group	≈ 1 ms hot; tens of ms in IA
Other readers	Athena, Trino, Spark, EMR, Snowflake read Iceberg (through a catalog)	only something that mounts it
Writers	conditional writes make concurrent Iceberg commits safe (the native engine probes it)	a hard link on POSIX (what Drishti uses on local disk)
Iceberg only: file paths in metadata	`s3://bucket/…` is the same for every task; Cross-Region copies read with `relocate`	the path depends on the mount; a moved folder is a moved table

At 200 TB+ EFS costs 10–40× more and shares one throughput budget, whatever the table format. **So the lake is in S3 for Iceberg too; EFS keeps the small, hot, per-node state.**

04
Iceberg in Drishti, as built

drishti-iceberg, the two connectors, catalogs, commits, S3 and its simulators, evolution, snapshots and time travel, maintenance, lake build to Iceberg

drishti-iceberg: Drishti's own Iceberg engine, no Hadoop

IcebergSourcePlugin · IcebergExternalPlugin · IcebergLoader · IcebergMaintenance	unchanged read paths
catalog/ path-based (`version-hint.text`), a metadata file named directly, REST (OAuth2, SigV4, vended credentials)	Jackson, JDK HTTP
meta/ table metadata v1/v2, schemas, partition specs, sort orders, snapshots, refs; unknown fields kept	Jackson
manifest/ manifest lists and manifests: data and delete manifests, partition summaries, column bounds	Avro
scan/ + read/ pruning by partition and bounds; Parquet by field id; position and equality deletes	parquet-java
write/ Parquet with field ids and metrics, manifests, commits, maintenance operations	no Hadoop Configuration
drishti-lakeio local disk (Windows paths, UNC), S3 (AWS SDK v2), ranged reads, codecs, footer cache	shared with Delta

0 Hadoop classes

loaded by connector, loader or maintenance (a test refuses them)

Windows

no winutils.exe, no HADOOP_HOME: load-iceberg.ps1, start-server.ps1 -Iceberg

Default since 2.6.0

engine: native; library kept for Azure, GCS, HDFS

Refused by name, never read wrongly: format version 3, encrypted tables, ORC and Avro data files, unknown types, S3 remote signing.

What of the Iceberg specification is covered

Area	Covered (BUILT)	Refused or later
Table metadata	every v1/v2 field; unknown fields kept when it writes the next version	format version 3 (refused)
Types	boolean, int, long, float, double, decimal, date, time, timestamp(tz), string, uuid, fixed, binary, struct, list, map	v3 types: timestamp_ns, variant, geometry
Schema evolution	by field id: add, drop, rename, reorder, int→long, float→double, decimal widening; name mapping	—
Partitioning	identity, bucket (Murmur3), truncate, year, month, day, hour, void; several specs (evolution)	—
Manifests	lists and manifests (Avro), data and delete manifests, inherited ids and sequence numbers, bounds	—
Deletes	position and equality deletes, by the sequence-number rules	v3 deletion vectors (Puffin): later
Snapshots	current snapshot, snapshot log (time travel by *known at*), the `main` ref	named branches and tags: later
Data files, encryption	Parquet	ORC, Avro data files; encrypted tables (refused)
Catalogs	path-based, a metadata file named directly, REST	Hive metastore, JDBC (only behind a REST catalog)

Two engines: native and library

engine	What reads and writes	Reaches	Use it for
`native` **(default)**	drishti-iceberg, no Hadoop	local disk (Windows too), `s3://`, `s3a://`; path, named metadata, REST	everything on disk and S3; Windows
`library`	Apache Iceberg Java 1.10.1 over Hadoop	the above plus `abfs://`, `gs://`, `hdfs://`	Azure, GCS, HDFS; a fallback
`auto`	native on Windows, library elsewhere		keeping the library on Linux

Measured (10,000 trades × 3 days)	native	library
load (30,000 rows)	22.7 s	25.7 s
connector start (plan, type-ahead)	637 ms	1,848 ms
first trade (cold: id map, footer)	212 ms	288 ms
one trade, document cache emptied: median / p95	54 / 111 ms	75 / 144 ms
a trade on a past date (day not yet read)	94 ms	353 ms

Why native is faster on first reads: each Parquet footer, manifest and delete file is read once and kept (data files never change). The library re-reads the footer on every single-entity read.
Caches: footers 50,000 row groups (shared with Delta), manifests 2,000,000 entries, delete rows 5,000,000.

Two connectors: Drishti's tables and anyone's tables

	`iceberg` (Drishti's tables)	`iceberg-external` (anyone's tables)
Written by	`IcebergLoader` in Drishti's layout	Spark, Flink, Trino, Snowflake, Glue, dbt, pyiceberg, any layout
The document	the `doc` column, exactly as loaded	**the row** as JSON (structs, lists, maps), or a JSON column
The id	the `id` column	one or several columns; by default the table's **identifier fields**
The business date	`business_date`	a date column, a timestamp's day in a zone, the partition spec's date; or current only
Searchable fields	the promoted columns the pack declares	**every primitive column** (nested to depth 2)
One entity	id map: one row group	sorted or bucketed by id: one row group; unsorted: an **id index** built on first use per day
Writes	loader and maintenance	**never**: the owners' engines maintain the tables
Deletes, time travel	position and equality deletes applied; *known at*	the same; branches and tags not read

Catalogs: what Drishti supports today

Catalog	How Drishti reaches it	Writes	Status
Path-based (`catalog: hadoop`, the default)	`<root>/<domain>/<table>/metadata/version-hint.text`; root a folder or `s3://`	loader and maintenance	**BUILT**
A metadata file named directly	`metadata-location.<kind>: s3://…/v12.metadata.json`	read-only	**BUILT**
REST: Polaris, Snowflake Open Catalog, Nessie, Lakekeeper, Unity, Gravitino	`catalog: rest`, `uri`, `warehouse`, OAuth2 `credential` or `token`	commits with requirements	**BUILT**
AWS Glue Data Catalog, through Glue's Iceberg REST endpoint	`uri: https://glue.<region>.amazonaws.com/iceberg`, SigV4 `signing-name: glue`	as REST	**BUILT** (not run on AWS)
AWS Glue Data Catalog API (GetTable, UpdateTable)	not implemented; use Glue's REST endpoint	—	**PROPOSED**
Amazon S3 Tables (table buckets)	not implemented; its Iceberg REST endpoint with SigV4 is the likely route	—	**PROPOSED**
Hive metastore, JDBC catalog	only through a REST catalog in front of them	—	**NOT BUILT**

```
settings: { catalog: rest, uri: "https://glue.us-east-1.amazonaws.com/iceberg", warehouse: "<account-id>",
            catalog.rest.sigv4-enabled: "true", catalog.rest.signing-name: glue, catalog.rest.signing-region: us-east-1 }
```

Commits and concurrency, as the code makes them

Local disk, EFS (Windows too)

write `vN+1.metadata.json` to a temporary name, then **create** it under its final name with a hard link (fails if taken); then replace `version-hint.text`

Tested: NativeCommitTest: 4 writers at once lose nothing

BUILT

Amazon S3, path-based

a **conditional PUT** (`If-None-Match: *`) of `vN+1.metadata.json`; a probe at the first commit; a store that ignores it makes the table **single-writer**

Tested: S3Mock, MinIO (racing writers on MinIO)

BUILT

REST catalog (Glue, Polaris)

the catalog's commit with requirements (`assert-table-uuid`, `assert-ref-snapshot-id`, ...); a **409** is re-read, re-applied, retried

Tested: REST protocol and catalog tests

BUILT

- **Two writers on one table:** one wins each version; the other re-reads, re-applies its change on the new version and retries (10 times)
- **A rewrite is validated:** a re-applied rewrite of a day fails, and is reported, if that day changed meanwhile; an append always re-applies
- **No orphans from Drishti's own writers:** files of a commit that finally failed are deleted at once

Same safety as Delta on S3, different writer. Delta's S3 commits come from `lake publish` (Python deltalake); Iceberg's from the Java loader itself. No DynamoDB lock table in either.

S3, and the simulators that prove it

Test	Simulator	What it proves	Runs on
`NativeOnS3SimulatorTest`	Adobe S3Mock in the JVM; MinIO binary	loader writes, connector reads an Iceberg table in S3; writers at once where atomic	Linux, Windows (S3Mock)
`NativeOnS3Test`	S3Mock image; MinIO image (Testcontainers)	the same; the commit probe decides single-writer	where Docker is reachable
`tools/lake/test_s3_simulators.py`	moto (pure Python); MinIO binary	**Delta**: table on S3, `lake publish` conditional PUT, two publishes at once	Linux, Windows (moto)
`deploy/aws/delta-lake/check-conditional-writes.sh`	your real bucket	the bucket refuses a second create (412): the probe both formats rely on	AWS (first run pending)

What the probe checks

- create-only (`If-None-Match: *`): the commit's create
- update-if-match and delete-if-match
- two racing creates: exactly one wins (atomic)
- each suite prints its line and skips the cases its simulator does not honour

Not yet proven on AWS: no Iceberg table has been written to a real S3 bucket or committed through a real Glue catalog.
First step of any Iceberg production plan: run `check-conditional-writes.sh` and the loader against a development bucket (PROPOSED).

Schema and partition evolution, as built

Read by field id

BUILT

renamed columns read correctly; added columns read as null; int→long, float→double, decimal widening; files without ids through `schema.name-mapping.default`

Add a promoted column

BUILT

the loader adds new promoted paths as columns; `IcebergMaintenance --columns trade:PATH` adds and fills them from the documents

Partition evolution (read)

BUILT

several specs in one table; every transform: identity, bucket, truncate, year, month, day, hour, void

Hidden partitioning (read)

BUILT

a table partitioned by `day(TRADE_TS)` or `bucket(id)` is pruned by its manifests; `iceberg-external` takes the business date from the spec

Drishti's writer changes a spec

PROPOSED

always `identity(business_date)`; no partition evolution written

Versus Delta in Drishti: adding a promoted column on Delta changes the table's columns, and `lake publish` then refuses the next day: the runbook writes a new folder (`trade_endur_v2`) and backfills.
On Iceberg the column is added in place by field id; old files read it as null until `--columns` rewrites them.

Why it matters for a bank: promoted columns grow as people search new fields. Iceberg absorbs that without a new table; Delta in Drishti's AWS flow needs a new table per change, because `lake publish` refuses a day whose columns differ.

Snapshots, time travel and deletes

A table's snapshot log, and what *known at* reads



A view *known at* that instant reads S3's schema and files; a later load is invisible; a time before the oldest snapshot kept reads nothing.

	Iceberg in Drishti	Delta in Drishti
Time travel unit	a snapshot (each commit), from the snapshot log	a table version (each commit), from the log
How far back	until snapshots are expired: `--expire-hours` (168 by default)	until vacuum and log retention (168 h; 7 years in the risk domains)
Rows deleted by other engines	position and equality deletes applied by **every** read, columns stay on	deletion vectors honoured; the table's searches fall back to documents
Drishti's own deletes	none: a restated day replaces its partition	none: a restated day replaces its partition
Format v3 deletion vectors	refused by name (later)	—

Maintenance as built: IcebergMaintenance

Option	What it does (one Iceberg commit each)	Status
`--keep-days N`	deletes business days older than the newest N on or before `--as-of`: metadata only; > 50 % of rows needs `--force-drop`	BUILT
`--relayout` (`--force`)	rewrites only days that drifted (overlapping id ranges, oversized files, delete files, missing columns), sorted, validated against the snapshot read	BUILT
`--columns kind:path,…`	adds promoted columns and rewrites every day, filling them from the documents	BUILT
`--rewrite-manifests`	regroups manifests, one per month of business dates	BUILT
`--expire-hours H`	expires snapshots older than H (168), keeps the current, deletes files only they referenced	BUILT
Orphan file removal	not in `IcebergMaintenance`: use Spark's `remove_orphan_files`, Athena `VACUUM`, Glue optimizers	PROPOSED
A UI, schedules, history (Admin → Lakes)	Delta only today (`lake maintain`)	PROPOSED
The 7-year floor of the risk domains	enforced by `lake maintain` for Delta; nothing stops `--expire-hours 168` on an Iceberg risk table	PROPOSED

Never apply S3 lifecycle **expiry** to a table's files, in either format: they delete files the metadata still names.

Lake build to Iceberg, and the demo's CALYPSO in Iceberg

1 JSON Lines in

`data/testdata/<domain>/<kind>/*.jsonl.gz` or a bank's dumps

2 lake build → Delta

staged under `<data>/build/iceberg-stage`: the same documents and promoted columns as a Delta build

3 IcebergLoader, engine native

each business day committed as one overwrite of its partition; no Maven, works on Windows

4 Iceberg table

`<data>/iceberg/trading/trade_calypso` (or `s3://…`)

5 connector file switched

`trade-calypso.yaml`: `plugin: iceberg`, `engine: native`, `table: {trade: trade_calypso}`

```
tools/load-iceberg.sh ./data/iceberg --trades 10000 --days 3          # samples and a book, one load
tools/load-iceberg.sh s3a://risk-lake/iceberg --set s3.region=us-east-1 --spill-dir /data/tmp
python tools/lake/stores.py --from data/delta/trading/trade_calypso --root data/iceberg --table trade_calypso --replace
```

Path to an Iceberg table	Scope	Status
`build_demo_lakes.py --store <system>=iceberg` (via **`stores.py`**)	the trading domain's trade tables	**BUILT**
`load-iceberg.sh` / **`load-iceberg.ps1`** → **`IcebergLoader`**	any kind, JSON Lines `{domain, kind, id, date, doc, columns}`	**BUILT**
`lake build --format iceberg` for any pack and kind (one command)	would replace the Delta staging step	**PROPOSED**

05

Built, designed, proposed

Every Iceberg capability with its label: what runs today, what a design document describes, and what this deck proposes

Built and planned (1/2): built

drishti-iceberg native engine: metadata, manifests, planning with pruning, field-id reads, deletes, time travel	BUILT
Writer: Parquet with field ids and metrics, manifests, commits (hard link, conditional PUT + probe, REST requirements)	BUILT
Catalogs: path-based, named metadata file, REST (OAuth2, ETag, vended credentials, SigV4 for Glue)	BUILT
`iceberg` connector: id maps, column sets, type-ahead, history days, BI scan, data versions, *known at*	BUILT
`iceberg-external`: rows as documents, identifier fields, every column searchable, id index, `tables: auto`	BUILT
`IcebergLoader` and `IcebergMaintenance` on both engines; `load-iceberg.sh` and `.ps1` (Windows)	BUILT
Mixed storage per source system (`--store`), with an equality test against all-Delta	BUILT
S3 via the AWS SDK; S3Mock and MinIO tests; TLS, private CA and proxy for the object store	BUILT
Shared AWS pieces: ECS images, EFS access points, Cloud Map, IAM roles, buckets (dry-run tested, from the Delta work)	BUILT

Built and planned (2/2): designed and proposed

Format v3 reading (deletion vectors, new types); named branches and tags	DESIGNED
Make a pack button for `iceberg-external` in Admin → Connectors (the two commands work today)	DESIGNED
VaR sharding across a cluster of Drishti servers (scatter-gather of partial vectors), on either format	DESIGNED
Iceberg plugin: `vectors()` for VaR and exposure, `documents()`, `tableIdentity()`, `inventory()`	PROPOSED
Official VaR and exposure results published to Iceberg (and to `s3://` at all)	PROPOSED
AWS scripts for an Iceberg lake: load, backfill, maintain, lifecycle, alarms, report	PROPOSED
Amazon S3 Tables and the Glue Data Catalog API as catalogs (REST endpoint already built)	PROPOSED
Orphan file removal; Admin → Lakes for Iceberg; the 7-year floor for Iceberg risk tables	PROPOSED
`lake build --format iceberg` for any pack; Iceberg → Delta conversion	PROPOSED

06

Deploying on AWS with Iceberg

The Delta deck's twelve setup steps, which are shared, which need an Iceberg variant; the catalog choice; IAM, KMS, the network, EFS and Drishti*net

Setup at a glance: steps 0 to 11, for an Iceberg lake

0 Prerequisites

VPC, S3 endpoint, EFS, KMS, ECS, ALB; plus a Glue endpoint if Glue

BUILT

1 Settings

`deploy/aws/env`; an Iceberg tables file

PROPOSED

2 Buckets

`setup-buckets.sh`: unchanged

BUILT

3 IAM

`setup-iam.sh`; Glue rights if a catalog

PROPOSED

4 Prove commits safe

`check-conditional-writes.sh`: unchanged

BUILT

5 Storage tiers

`setup-lifecycle.sh`: Iceberg prefixes

PROPOSED

6 EFS access points

`ecs/setup-efs.sh`: unchanged

BUILT

7 Cloud Map seeds

`ecs/setup-cloud-map.sh`: unchanged

BUILT

8 Images

the server image carries `IcebergLoader`

BUILT

9 Tasks, services

`ecs/register.sh` + `DRISHTI_ICEBERG_ROOT`

PROPOSED

10 First load

an Iceberg `load-day.sh`

PROPOSED

11 Alarms and report

Iceberg metrics: snapshots, manifests

PROPOSED

The catalog choice on AWS

Path-based tables in S3

BUILT

- no catalog service: `metadata/version-hint.text` in the table folder
- commits: conditional PUT, probe at first commit
- other engines read by naming the metadata file, or after registering the table in Glue
- **best when** only Drishti writes and Drishti reads

AWS Glue Data Catalog (Iceberg REST endpoint)

BUILT

- `catalog: rest`, SigV4 `signing-name: glue` (client built; not yet run on AWS)
- commits through the catalog (409 retried); Athena, EMR, Redshift see the same tables
- IAM (and Lake Formation, if used) governs every table
- **best when** Athena or Spark also read or write

Amazon S3 Tables (table buckets)

PROPOSED

- not in Drishti's code or docs
- exposes an Iceberg REST endpoint with SigV4: the likely route through the existing REST client
- AWS runs compaction and snapshot management itself
- **prove first** in a development account

Recommendation (PROPOSED): start path-based (built, no extra service, the same commit safety as Delta). Move a table behind Glue's REST endpoint when another engine must write it or Athena must query it; the connector's name, kinds and packs do not change, only `catalog`, `uri` and the signing settings.

IAM, KMS and the network for Iceberg

Role	Delta lake (built policies)	Iceberg lake (PROPOSED changes)
server task	GET, LIST on `lake/`; `kms:Decrypt`	the same on `iceberg/`; with Glue: `glue:GetCatalog`, `GetDatabase`, `GetTable` (read)
loader task	PUT data files and commits; **never delete** (explicit deny)	PUT data, manifests, `vN.metadata.json`, `version-hint.text`; the deny can stay (old metadata removal is best effort); no `--keep-days` in the loader; with Glue: `glue:UpdateTable`, `CreateTable`
maintenance task	retention, checkpoints, vacuum: the only role that deletes	`--keep-days`, `--expire-hours`, `--relayout`: deletes data, manifests, old metadata
KMS	SSE-KMS bucket default, S3 Bucket Keys	unchanged: the native writer sends no SSE header; bucket default applies. Iceberg table encryption: refused

Network

- S3 gateway endpoint, bucket policy allowing only it (as Delta): unchanged
- Path-based tables need nothing else
- Glue catalog: an interface endpoint for Glue (PROPOSED), or traffic to the Glue endpoint through the proxy settings
- Vended credentials: STS endpoint (already among the Delta prerequisites)

The proxy and TLS settings carry over: `proxy.*` and `tls.*` apply to the REST catalog's client and to the object store (ICEBERG_CONNECTOR.md: Security; Outbound proxy).
Private CA for the catalog's own HTTPS: import into the JVM truststore.

EFS, identical paths and Drishti*net: unchanged

drishti-a, drishti-b

- `trade-endur` (Delta) and `trade-calypso` (Iceberg)
- replicas of each other for both

drishti-c, drishti-d

- `trade-murex` (Iceberg, native)
- a shard: searches ask it too

S3 lake bucket

- `lake/` (Delta tables) and `iceberg/` (Iceberg tables) side by side; every node can read every table

EFS (identical paths)

- `/drishti/<env>/<node>/opt/drishti/{data,config,sutras}`: connector files, users, Drishti*net state
- loader: spill files of the external sort and the per-table locks

Same cluster rules. Nodes that read the same tables are replicas; different tables of a kind are shards; a failed node costs speed, never answers.

One gap: the Iceberg plugin reports no table identity, so two nodes on one Iceberg table are not recognised as replicas (a search may ask both). PROPOSED: the table's UUID and location.

Storage tiers by business date, for Iceberg prefixes


```
{"ID": "tier-iceberg-trading-trade_endur-2026-03",
 "Status": "Enabled",
 "Filter": {"And": {
   "Prefix": "iceberg/trading/trade_endur/data/business_date=2026-03",
   "ObjectSizeGreaterThan": 131072}},
 "Transitions": [
   {"Date": "2026-06-29T00:00:00Z", "StorageClass": "STANDARD_IA"},
   {"Date": "2028-03-30T00:00:00Z", "StorageClass": "GLACIER_IR"}]}
```

- Same rule shape, same dates, same tiers as Delta
- Prefix: `…/<table>/data/business_date=YYYY-MM`
- `metadata/` never matches: metadata JSON, manifest lists and manifests stay in Standard

PROPOSED

0 – 90 days

S3 Standard

90 days – 2 years

Standard-IA

2 years – kept days

Glacier Instant Retrieval

beyond

removed by `--keep-days` → `--expire-hours`

Iceberg-specific risks: a manifest still names every data file of an old day, so **never** an S3 expiry on table files (the table breaks); and Glacier Flexible Retrieval or Deep Archive would make a planned read wait hours. Only `IcebergMaintenance` removes files, then S3 keeps noncurrent versions 14 days.

07
The layout

Prefixes, the table, its columns and properties, and the partition: what goes where in S3, and why

Prefixes and tables


```
s3://acme-drishti-prod-lake/
├── lake/                    Delta tables (DRISHTI_DELTA_ROOT)
└── iceberg/                 DRISHTI_ICEBERG_ROOT = s3://…/iceberg
    └── trading/             the domain (a REST catalog's namespace)
        ├── trade_calypso/   one table per source system and kind
        │   ├── metadata/
        │   │   ├── v12.metadata.json   schema, spec, sort order,
        │   │   │                       properties, snapshots
        │   │   ├── version-hint.text   path-based tables only
        │   │   ├── snap-…-….avro       a manifest list per snapshot
        │   │   └── …-m0.avro           manifests: files, partitions,
        │   │                           column bounds
        │   └── data/
        │       ├── business_date=2026-10-19/
        │       └── business_date=2026-10-20/
        │           ├── 2026-10-20-00000-<uuid>.parquet  ids A… – C…
        │           └── 2026-10-20-00001-<uuid>.parquet  ids C… – Z…
        └── trade_murex/ …
```

Choice	Why
One table per source system and kind	each source loads, restates and fails on its own (`table.trade: trade_calypso`)
Domain as the first folder	the connector's `domain`; IAM and lifecycle can name it
`metadata/` beside `data/`	the whole table under one prefix: copy, replicate, grant as a unit
Absolute paths in metadata	a moved table needs `relocate` (native engine)

Columns and table properties

Column (field id)	Type	Content
`id` (1)	string, required	the entity's id: `MX-20000001`
`doc` (2)	string	the whole JSON document, exactly what a view shows
`business_date` (3)	date, required	the identity partition
promoted (`TRADE_ID`, `PORTFOLIO`, …)	string / double	the pack's `layout.<kind>.columns`; `LEGS.CURRENCY` → `LEGS__CURRENCY`

Property	Value	Why
partition spec / sort order	`identity(business_date)` / `id ASC`	a day is a unit; engines that honour the sort order keep it
`format-version`	2	position and equality deletes
`write.parquet.row-group-size-bytes`	1,048,576	one read decodes about 3 MB of documents (sized compressed)
`write.parquet.compression-codec`	zstd	about 0.7 KB per 6.5 KB trade (measured)
`write.metadata.metrics.column.doc` / `.id`	none / full	no bounds of documents; exact id bounds per file
`write.metadata.previous-versions-max`	50 (delete after commit)	old metadata files removed
`commit.retry.num-retries`	10	concurrent day commits retry
`drishti.layout.file-rows`	250,000	rows per file; maintenance re-cuts drifted days

Partitions, and what each read needs

Key	Delta on AWS (built)	Iceberg (built)	Why
business_date	first partition key	the only partition (identity)	every interactive read names a date; a day is loaded, restated, dropped as a unit
PRODUCT_TYPE	second key (`--partition-by`)	not a partition	Delta needs it to sort a partition in one batch; Iceberg sorts a whole day externally
Hidden transforms	—	read for any table (`day(ts)`, `bucket(id)`)	other engines' tables, served by `iceberg-external`

Need	What it reads	So the table needs
One entity	the day's id map, then one row group of one file	rows sorted by id; small row groups
History of one entity	the layout's days whose id map holds it	`business_date` as the partition
Search, pick lists, impact	the day's column set: id and promoted columns, never `doc`	every searched field promoted
File and row-group skipping	manifest bounds per file; footer statistics per row group	metrics on id and promoted columns, none on `doc`

08
Files and loading

Sizes, row groups, sorting, one commit per day, the daily load on AWS, backfill and restatement

The numbers, Delta and Iceberg

Setting	Delta on AWS	Iceberg	Why it differs
Rows per file	250,000 (`file-rows`)	250,000 (`--file-rows`)	—
Row group	1 MB (`LAKE_ROW_GROUP_KB`)	1 MB **compressed** (`--row-group-mb`): 300–700 trades	Iceberg sizes by compressed bytes
Compression	Snappy	zstd (about 9× on documents)	the writers' choices
Statistics	log: id, business_date, promoted	manifests: id full, promoted, none on `doc`	—
Sort	by id within a batch (`LAKE_BATCH_ROWS` 250,000)	by id over the whole day (external sort, spill)	no partition-size limit
Commit unit	one per table per day (`lake publish`)	one per day of ≥ 100,000 rows; small days together	fewer commits for small tables
History kept	log 30 days, files 7 days (7 years in risk)	snapshots 168 h (`--expire-hours`); metadata 50 versions	see section 13

Sorted by id, one commit per day

read
- JSON Lines from a file or stdin; dates after tomorrow refused

→

sort
- per day, in memory up to `--buffer-mb`, spilled runs beyond

→

merge
- runs merged in id order; files of 250,000 rows

→

commit
- one overwrite of the day's partition: readers see the old day or the new

- A file holds a contiguous id range; manifests keep each file's min and max `id`; footers each row group's: one id reads one row group
- Sorting has **no batch limit** (unlike `lake build`'s 250,000 rows): the spill directory needs about the JSON's size
- A day is replaced whole: the load is idempotent; *known at* before the commit still sees the old day
- Another engine (Spark) writes the same layout: `repartitionByRange("id").sortWithinPartitions("id")`, `WRITE ORDERED BY id`

≈ 40 s a million

a day of 1,000,000 trades written by one thread: 4 files, 0.70–0.75 GB (estimate from a trial)

17,910 rows in 8 s

the samples into 46 tables, measured; 30,000 trade-days in 5 s

The daily load on AWS: Delta's four steps, Iceberg's one

Delta (built): build locally, then publish

aws s3 sync

- landing → EFS scratch

↓

lake build

- scratch Delta table

↓

aws s3 sync

- data files → S3

↓

lake publish

- one commit, conditional PUT

Iceberg (PROPOSED script): the loader writes S3

IcebergLoader, engine native

- reads the landing dumps (stdin or a file)
- sorts each day by id, spills to EFS or ephemeral disk
- writes Parquet and manifests to `s3://…/iceberg/<domain>/<table>/`
- commits: conditional PUT of `vN+1.metadata.json`, then `version-hint.text`

```
aws s3 cp s3://…-landing/2026-10-20/calypso/ - \
    --recursive … | java \
  -Dloader.main=…iceberg.IcebergLoader \
  -jar drishti-server-exec.jar \
  - s3://…-lake/iceberg --engine native
```

Fewer moving parts on Iceberg: no scratch table and no second sync; the loader holds the table's lock only while it writes. Keep the Delta scripts' rules: one table per lock, `--dry-run`, exit codes, metrics to `Drishti/Lake`.

Backfill and restatement

A source re-sent	Delta (built scripts)	Iceberg (loader built; script PROPOSED)
some whole days	`backfill.sh --dates 2026-10-14,2026-10-15`	load the days again: each replaces its partition in one commit
one product type of a day	`load-day.sh … --mode overwrite-partitions`	the whole day again (product type is not a partition)
years of history	`backfill.sh --from … --to …`, oldest first, resumable	one load per range; days of ≥ 100,000 rows commit as written
old days to drop	`lake maintain` retention	`IcebergMaintenance --keep-days` (metadata only)

- Readers switch at the commit; a reader sees the old day or the new, never a mixture (both formats)
- *Known at* before the restatement shows the day as it was until snapshots expire (Iceberg) or vacuum runs (Delta)
- Data files of an old restated day tier by their date prefix at the next lifecycle run (the rule is by date, not object age)

Guarded: removing more than half of a table's rows fails, removing nothing, unless `--force-drop`; a row dated after tomorrow in the business zone is not loaded and the load ends with an error naming it.

09
Maintenance

What a nightly Iceberg maintenance task would run, why snapshots must be expired, orphan files, and what managed platforms do instead

What a nightly Iceberg maintenance run does


```
java -cp <server exec jar> com.ash.drishti.plugin.iceberg.IcebergMaintenance s3://…/iceberg \
     --domain trading --keep-days 1800 --relayout --rewrite-manifests --expire-hours 168
```

PROPOSED

Step	Why	Delta's counterpart
`--keep-days 1800`	drop business days older than seven years: metadata only	retention (`keep-business-days`)
`--relayout`	re-sort days another writer appended out of order; fold delete files into data	compaction / relayout
`--rewrite-manifests`	one manifest per month: planning reads a few dozen manifests, not thousands	checkpoint
`--expire-hours 168`	keep the metadata JSON small; bounds *known at*	vacuum + log retention
orphan files	files of a crashed writer no snapshot names (not built: use Spark, Athena or Glue)	vacuum

160 ms → 33 ms

connector start on 1,800 daily commits, before and after manifests + expiry (measured)

93 ms a commit

writing a table of 1,800 daily commits (measured, small rows)

0.7 s / 2.9 s

drift check / keep 5 + rewrite 5 + manifests + expiry, 10,000 trades

10

Caches and staying fast

What each server keeps, how Iceberg keys it, what never scans the whole lake, and the numbers to watch

What each server caches, Delta and Iceberg

Cache	Delta	Iceberg	Keyed by
the table's file list / layout	heap, from the last checkpoint	heap, per snapshot (`layout-cache` 64)	Delta: version; Iceberg: snapshot
id maps of days	heap (`id-map-mb`) **and disk** (memory-mapped)	heap (`id-map-mb`, 1024)	Delta: table version; **Iceberg: the day's fingerprint**
column sets of days	heap (`columns-cache-mb`)	heap (`columns-cache-mb`, 1024)	as above
documents	heap **and RocksDB on disk**	heap (`doc-cache-mb`, 256)	as above
Parquet footers	heap (native engine), shared	heap (native engine), shared	the file (never changes)
manifests, delete files	—	heap: 2,000,000 entries; 5,000,000 delete rows	the file

Iceberg's edge: committing a new day leaves every other day's caches in place (keyed by the day's files). On Delta a commit moves the table version, so untouched days reload.

Delta's edge: id maps and documents survive a restart on disk (EFS or NVMe). PROPOSED for Iceberg: the same disk caches, keyed by the day's fingerprint.

What never scans the whole lake

Request	Reads (Iceberg)	Grows with history?
Open one entity on a date	the planned layout (in memory), the day's id map, one row group of one file	no
History of one entity over N days	the layout's days whose id map holds it	with N
Type-ahead	the newest day's ids (`index.days` more if set)	no
Search, pick list, impact on a date	that day's column set; files and row groups skipped by bounds	no
Refresh	`version-hint.text` (path) or `loadTable` with ETag (REST); a changed snapshot's manifests	with live manifests: regroup monthly
Load, restatement	one day's files and one commit (metadata JSON rewritten)	metadata JSON grows with snapshots kept
Questions over all of history	not interactive: Athena, Trino, Spark on the same tables	—

The one Iceberg-specific growth: each commit rewrites a metadata JSON that lists every snapshot kept. Expire snapshots (or, for seven-year risk tables, see section 13) and regroup manifests monthly.

The numbers to watch, for an Iceberg table


```
{"name": "trade-calypso", "health": "UP", "engine": "native",
 "cache": {"tables": 1, "snapshots": 1, "idMaps": 2, "columnSets": 2, "documents": 2,
           "partitions": 0, "daysWithDeletes": 0, "ids": 10000}}
```

Number	Healthy	If not	Status
`daysWithDeletes`	0 for Drishti's tables	single reads slow: `IcebergMaintenance --relayout`	**BUILT**
Health: `not laid out as the pack declares`	absent	`--columns trade:path,…` or reload	**BUILT**
`DEGRADED: … names files that are not there`	absent	a moved table: set the `relocate` pair Health gives	**BUILT**
snapshots kept, metadata JSON size	bounded by `--expire-hours`	commits and refreshes slow: expire	**PROPOSED** metric
live manifests per table	a few dozen	planning slow: `--rewrite-manifests`	**PROPOSED** metric
single-writer warning in the log	absent	the store ignores `If-None-Match`: one writer per table	**BUILT** (log)

11

Delta Lake and Iceberg side by side

Transaction log against metadata tree; commits on S3; time travel, evolution, partitioning, deletes; maintenance, catalogs, the ecosystem and the running cost

Inside each format: a log against a tree

Delta Lake: a transaction log

table folder

- `s3://…/lake/trading/trade_endur/`

_delta_log/

- `…0451.json` `…0452.json` one commit each: add / remove + stats
- `…0450.checkpoint.parquet` every 10 commits
- `_last_checkpoint`: where to start replaying

data files

- `business_date=…/PRODUCT_TYPE=…/part-….snappy.parquet`

Read: replay checkpoint + later commits → the file list. Commit: create the next `NNN.json`.

Iceberg: a metadata tree

catalog pointer

- `version-hint.text` (path) or the REST catalog's metadata location

vN.metadata.json

- schemas, partition specs, sort orders, snapshots, properties

manifest list (per snapshot)

- `snap-….avro`: the manifests, with partition summaries

manifests

- `…-m0.avro`: data and delete files, partition values, column bounds

data and delete files

- `data/business_date=…/…parquet`; position or equality delete files

Read: pointer → metadata → one manifest list → pruned manifests. Commit: create `vN+1.metadata.json` (or the catalog swaps the pointer).

Side by side (1/3): log, commits, time travel, evolution

Aspect	Delta Lake	Apache Iceberg	In Drishti today
Where the truth is	an ordered log of JSON commits + Parquet checkpoints	a tree: metadata JSON → manifest list → manifests	both read natively, no Hadoop (BUILT)
Commit on S3	create `_delta_log/N.json` if absent	create `vN+1.metadata.json` if absent, or a catalog swap	both by `If-None-Match: *` (BUILT); Delta via `lake publish`, Iceberg via the Java loader
Concurrency	optimistic; conflicting writers retry	optimistic; requirements re-validated	a rewrite of a changed day is refused (Iceberg, BUILT); per-table locks in the scripts (Delta)
Time travel	by version or timestamp, until vacuum and log cleanup	by snapshot or timestamp, until snapshots expire	*known at* on both (BUILT); 7-year floor for Delta risk tables only
Schema evolution	add columns by name; renames need column mapping	by field id: add, drop, rename, reorder, widen	Iceberg reads every change by id (BUILT); Delta publish refuses a changed day
Reading cost of a refresh	commits since the last checkpoint	one small pointer, then metadata if changed	Iceberg keys caches by day (BUILT)

Side by side (2/3): partitions, deletes, maintenance, catalogs

Aspect	Delta Lake	Apache Iceberg	In Drishti today
Partitioning	Hive-style folders on columns; liquid clustering (Databricks)	**hidden**: transforms (day, bucket, truncate) in the spec; specs evolve	both: `business_date` only (+ PRODUCT_TYPE on Delta/AWS); Iceberg transforms read (BUILT)
Row deletes, merges	deletion vectors or file rewrites (copy-on-write)	position / equality delete files (merge-on-read) or rewrites	Drishti writes neither: a day is replaced; reads others' deletes (Delta: columns off; Iceberg: columns on)
Maintenance	OPTIMIZE, checkpoint, VACUUM	rewrite data and manifests, expire snapshots, remove orphans	Delta: `lake maintain` + Admin → Lakes; Iceberg: `IcebergMaintenance` CLI, no orphan removal
Catalog	optional (the log is the table); Unity, Glue for discovery	central to most engines (Glue, Polaris, Unity, Nessie); path tables work	Delta: none; Iceberg: path or REST, incl. Glue REST (BUILT, not run on AWS)
Table identity across nodes	table id in the log	table UUID in metadata	Delta reports it to Drishti*net; Iceberg not yet (PROPOSED)

Side by side (3/3): the ecosystem and the running cost

Engine	Delta Lake	Apache Iceberg
Apache Spark, Amazon EMR	read and write (Delta's Spark library)	read and write (Iceberg's Spark runtime)
Trino, Starburst	Delta connector	Iceberg connector
Amazon Athena	reads Delta tables	reads and writes Iceberg tables in the Glue catalog; OPTIMIZE, VACUUM
Snowflake	reads through external tables / catalog integration	native Iceberg tables (managed or catalog-linked), Open Catalog (Polaris)
Databricks	native (Unity Catalog)	through Unity's Iceberg REST interface or UniForm
pyiceberg, DuckDB, pandas	delta-rs (deltalake), DuckDB's delta extension	pyiceberg, DuckDB's iceberg extension (both used in Drishti's tests)

Running cost item	Delta	Iceberg
Storage (Parquet), tiers	same	same
Metadata per commit	a small JSON; a checkpoint every 10	a metadata JSON listing every snapshot kept + a manifest list + a manifest
Catalog	none	none (path) or Glue / S3 Tables request and storage charges
Maintenance compute	nightly `lake maintain`	nightly `IcebergMaintenance`, or managed optimizers (charged)

Drishti's support, side by side

Capability	Delta	Iceberg
Read local disk, EFS, S3 without Hadoop; Windows	**BUILT**	**BUILT**
Write a table in S3	**BUILT** (`lake publish`)	**BUILT** (the loader, directly)
Tables other engines write, in their own layout	**BUILT** (Delta connector)	**BUILT** (`iceberg-external`, read-only)
REST or Glue catalog	— (not needed)	**BUILT** (client; not run on AWS)
AWS scripts: load, backfill, maintain, lifecycle, alarms, report	**BUILT** (dry-run tested)	**PROPOSED**
Maintenance UI, schedules, ownership rule, 7-year risk floor	**BUILT**	**PROPOSED**
Disk caches (id maps, documents) across restarts	**BUILT**	**PROPOSED**
VaR and exposure vectors (`list<double>`, `list<float>`)	**BUILT**	**PROPOSED**
Official VaR and exposure results published	**BUILT** (local or EFS root)	**PROPOSED**
Schema change without a new table	**PROPOSED**	**BUILT**
Caches survive a commit of another day	**PROPOSED**	**BUILT**

12

Which to choose

A decision guide for a bank, each statement tied to what Drishti supports today

A decision guide for a bank

If the bank ...	Choose	Because (Drishti today)
runs VaR or exposure in Drishti from the lake	**Delta** for the risk domain	the engines read vectors only through the Delta plugin; official results publish to Delta
wants the AWS scripts as they are (load, backfill, maintain, tiers, alarms)	**Delta**	the scripts and runbooks exist and are dry-run tested; Iceberg's are proposed
already keeps trades in Iceberg (Snowflake, Glue, Spark pipelines)	**Iceberg**, read in place	`iceberg-external` serves them read-only; no copy, no Drishti writer
needs other engines to write the same tables Drishti reads	**Iceberg** behind a REST catalog	commits through the catalog's requirements; Drishti's loader is one writer among several
changes promoted columns often	**Iceberg** for those tables	a column is added by field id; Delta's publish needs a new table
standardises on Databricks and Unity Catalog	**Delta**	Drishti reads Delta natively; Iceberg only through a REST catalog
must run Drishti on Windows laptops for QA	either	both native engines run without Hadoop

The recommendation, in one page

Today: Delta for Drishti-owned risk, Iceberg where the bank's data already is

- **Risk domain** (P&L vectors, scenario PVs, exposure paths, official results): Delta, on the built AWS scripts
- **Trades, reference, market data**: either, chosen per source system (`--store`)
- **The bank's Iceberg tables** (Snowflake, Glue): `iceberg-external`, read in place
- **One switch per table**: the connector name never changes, so a table can move later

To make Iceberg a full equal (PROPOSED)

1. `vectors()` in the Iceberg plugin, and the loader writing `list<double>` / `list<float>` columns
2. official results published to Iceberg (and to `s3://` for both formats)
3. AWS scripts: an Iceberg `load-day`, `maintain`, lifecycle prefixes, alarms, report
4. orphan removal, Admin → Lakes and the 7-year floor for Iceberg
5. `tableIdentity()` for Drishti*net replicas; disk caches
6. a first real run on AWS S3 and on Glue's REST endpoint

13

Risk workloads on Iceberg

VaR and exposure vector tables, official results, and sizing at 1.3 million positions and 100,000 portfolios: where the table format matters

VaR and exposure inputs: what each needs from the format

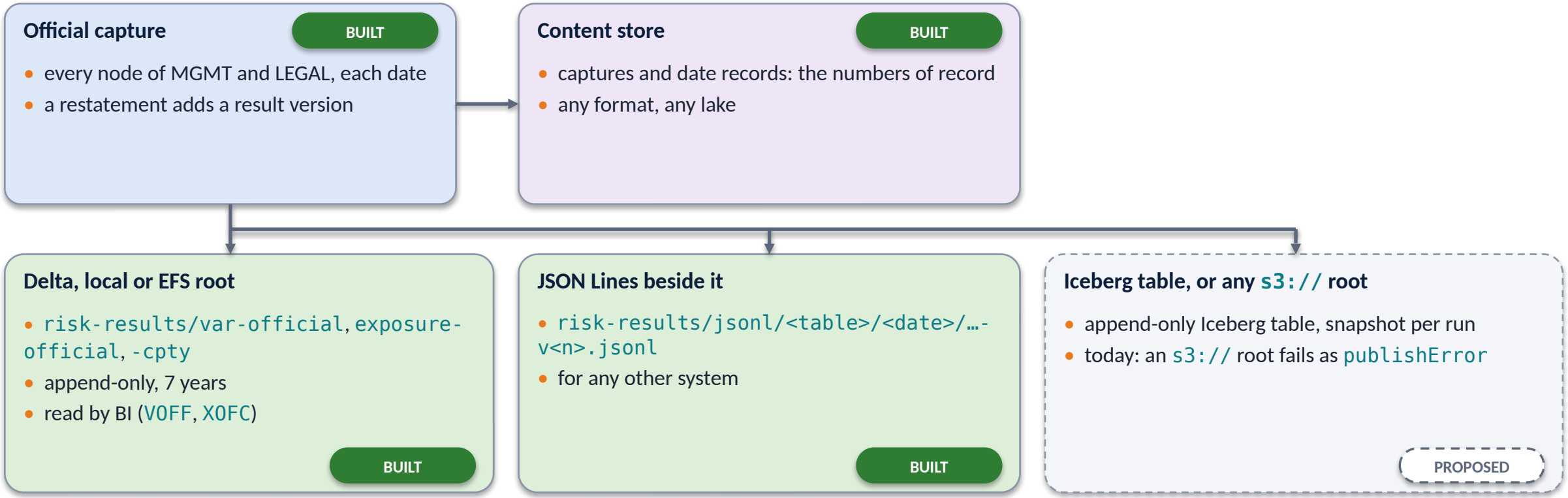
Input (pack)	Vector column	Size driver	Delta	Iceberg
`pnl-vector` (market-risk)	`PNL list<double>`, 509 scenarios	one row per trade per date	**BUILT**	**PROPOSED**
`scenario-pv` (market-risk, 2.9.4)	scenario PVs, `list<double>`	one row per trade or position per date	**BUILT**	**PROPOSED**
`exposure-path` (counterparty-risk)	`MTM list<float>` or `list<double>`, paths	about 4,000 × VaR's vectors	**BUILT**	**PROPOSED**
`scenario-set`, **`risk-hierarchy`**, **`portfolio`**	none	small tables	**BUILT**	**BUILT** (plain kinds)

What is missing for Iceberg, and how small it is

- **Engine:** list types read and written by drishti-iceberg (BUILT)
- **Loader:** `layout.<kind>.vectors` → a `list<double>` column, the document keeping `{"$vector": "PNL"}` (PROPOSED)
- **Plugin:** `vectorColumns()` and `vectors()` streaming a date file by file, one reused buffer (PROPOSED)
- **Metrics:** none on vector columns, as on `doc` (PROPOSED)

Why the format barely matters for the arithmetic: the engine reads a date once into its memory-mapped hot file and answers from there (VaR deck slide 10). The lake is read at load: 5.29 GB of float64 a date at 1.3M positions, either format the same Parquet column.
Where it does matter: commits, history and metadata (next slides).

Official results: published to Delta only, today



Honest status: official VaR and exposure results are published to a **Delta** table on a local or EFS root, plus JSON Lines. Not to Iceberg, and not to `s3://` in either format. On AWS today: publish to EFS and copy, or read the JSON Lines; an Iceberg (or S3) publisher is PROPOSED.

Sizing at bank scale: where the format matters

Per business date (1.3M positions, 509 scenarios)	Value	Same on both?	Basis
P&L or scenario-PV vectors, float64	5.29 GB raw, ≈ 4 GB Parquet	yes: one `list<double>` column	arithmetic; ≈ 0.75 × raw (estimate)
A year / seven years of vectors	≈ 1 TB / ≈ 7 TB	yes	estimate
100,000 portfolio vectors (if stored)	407 MB a date a currency	yes	arithmetic; storing them is proposed
Full-run commits	252 a year, one per date	yes	one commit per date
Intraday micro-batch commits	one per window per table	Delta: one JSON each; Iceberg: a snapshot each	depends on the window
Seven years of time travel	≈ 1,800 full-run versions + intraday	**no**: see the next slide	VaR: 7 years kept (decided)
Files a date	≈ 8–16 at 250–512 MB	yes	estimate; layout open (VAR_GUIDE 29.2)

The data volume is format-neutral; the history is not. Both formats store the same Parquet; they differ in how they keep seven years of versions.

Seven years of time travel: Delta against Iceberg

	Delta (risk domain, built)	Iceberg (proposed for risk)
How 7 years are kept	log and removed files kept 2,557 days; checkpoints	keep every snapshot 7 years (`--expire-hours` ≥ 61,368)
Cost per commit	a small JSON; replay from the last checkpoint	the metadata JSON rewritten with every snapshot kept
Measured	7,200 files list in 84 ms from a checkpoint	1,800 daily commits: 93 ms a commit, start 160 ms → 33 ms after regrouping
With intraday micro-batches	more small JSON files; checkpoints absorb them	more snapshots in every metadata file: compact intraday into the day's run
Guard against a short setting	`lake maintain` raises it (`retention-floor`)	none today (PROPOSED)

PROPOSED for an Iceberg risk lake: keep full-run snapshots for seven years and expire intraday snapshots after the day's official run (the content store keeps the official captures); add a 7-year floor to `IcebergMaintenance`; measure the metadata JSON at 2,000+ snapshots before committing.

14

Migration and coexistence

Delta and Iceberg side by side, converting tables between them, and keeping one logical connector name across a switch

Running Delta and Iceberg side by side

kind `trade` (trading pack) · one search, one page, one BI model over all three

trade-endur

- plugin: delta · lake/trading/trade_endur

trade-calypso

- plugin: iceberg · iceberg/trading/trade_calypso

trade-murex

- plugin: iceberg · iceberg/trading/trade_murex

Compared, all-Delta against mixed (stores_equality.py)	Result
six trade pages; `TRD <id> 10` histories for each	the same
two counterparty-trades pages; three searches	the same
a BI model query	the same
total	**18 answers the same**

Coexistence is built. One pack, one route, connectors of either format behind it; the BI column scan, data versions and held dates work on both (2.6.7).
Each table has exactly one connector: never both formats for the same system's trades.

Converting tables between formats

Direction	How	History	Status
Delta → Iceberg (trade tables)	`stores.py` / `build_demo_lakes.py --store <system>=iceberg`: rows → `IcebergLoader`	current days; one snapshot per copied day	**BUILT**
Delta → Iceberg (any kind)	the same streaming, generalised to any pack and domain	current days	**PROPOSED**
Iceberg → Delta	export documents as JSON Lines, then `lake build` (and `lake publish` on S3)	current days	**PROPOSED**
In place: Delta UniForm, Iceberg `snapshot`/`migrate`	other engines write the second format's metadata over the same Parquet	the converter's choice	**PROPOSED** (evaluate)

Keep the old table until the new one is proven: build the Iceberg table beside the Delta one, run the equality test against both, then switch the connector file. Keep the Delta folder for its time travel (and the content store keeps every official capture), or archive it.
Never let two connectors serve one system's trades at once: the union would double them.

Keeping one logical connector name across a switch

1 Build beside

write the Iceberg table next to the Delta one (`stores.py`, or the loader)

2 Compare

`stores_equality.py`-style answers, both lakes, the same dates

3 Switch

edit `trade-calypso.yaml`: `plugin: iceberg`, `engine: native`, `root`, `table.trade`, the same `layout`

4 Signal

the data-load signal, or wait `refresh-seconds`; Health shows `engine: native`, UP

5 Keep, then retire

keep the Delta table for its history; roll back by restoring the file

```
# config/connectors/trade-calypso.yaml (the name never changes)
plugin: iceberg                 # was: delta
kinds: [trade]
settings:
  domain: trading
  engine: native
  root: ${DRISHTI_ICEBERG_ROOT:${drishti.data.dir:./data}/iceberg}
  table: {trade: trade_calypso}
  layout: {trade: {columns: [TRADE_ID, SOURCE_SYSTEM, …]}}
```

Nothing above the connector moves: the pack's route, saved searches, BI models, reports and bookmarks name the connector or the kind, never the format.
Caches of the old connector are dropped when the file changes.

15

Sizing and cost

What changes from the Delta deck's 200 TB bill when the tables are Iceberg (list prices, one Region, estimates)

Monthly cost: what changes with Iceberg

Item	Delta deck (200 TB, per month)	With Iceberg
S3 storage, tiered	about USD 1,400	the same tiers; zstd may store less than Snappy (not measured side by side)
S3 requests	USD 500–700	about the same for reads; a few more small objects per commit (metadata JSON, manifest list, manifest)
Catalog	none	none (path-based) · Glue or S3 Tables: their request and storage charges
Server nodes	USD 650–750 (3 tasks)	the same tasks
Loader, maintenance	USD 50–150	fewer steps (no scratch table, no sync); or managed optimizers, charged separately
EFS	USD 300 + throughput	the same; no disk caches for Iceberg today, so less EFS I/O and colder restarts
Monitoring, secrets, endpoints, ALB	USD 200–400	+ a Glue interface endpoint if Glue is the catalog

The bill is the same order of magnitude in either format: storage dominates, and it is the same Parquet in the same tiers. The choice is about capability and operations, not cost.

16
Runbooks

The daily load, a restatement, a slow table, a moved or replicated table, and the warnings to act on

Runbooks: symptoms, causes, fixes

Symptom	Likely cause	Fix
`DOWN: cannot reach …`	root, bucket or catalog namespace not reachable	check `root` / `uri`, the task role, `s3.*`
`DOWN: no Iceberg tables under …`	nothing loaded in that domain, or the wrong `domain` / `namespace`	load it; check the domain
new data not visible	the refresh has not run yet	wait `refresh-seconds` (10) or send the data-load signal
single reads slow, `daysWithDeletes` **> 0**	another engine's equality deletes	`IcebergMaintenance --relayout`
single reads slow on another engine's table	large row groups or unsorted days	`--relayout --force`, or that engine's sorted rewrite
searches say `partial: true`	a field the query reads is not promoted	add it to the layout, then `--columns`
***known at* finds nothing**	the snapshots of that time were expired	keep more history (`--expire-hours`)
log: `… does not honour conditional writes … single-writer`	the store ignores `If-None-Match`	one writer per table, or a REST catalog
loader: `iceberg: the load failed: <why>`	the first reason in the failure's chain	fix and load the day again (idempotent)

Disaster recovery and moved tables

Event	How to recover (Iceberg)	Status
A wrong commit	*known at* reads the snapshot before it; restate the days from the landing bucket	**BUILT**
Files deleted by mistake	S3 versioning: restore noncurrent versions within 14 days (as Delta)	**BUILT** (bucket)
A table moved or copied (a laptop, another bucket)	`relocate: '<old> -> <new>'`, the pair Health gives; the library engine ignores it	**BUILT**
A Region lost	CRR of the lake bucket; read the replica with `relocate: 's3://prod/… -> s3://dr/…'` once the backlog is empty	**PROPOSED** (test)
A commit arrived before its files (replication)	that day fails to read until the files arrive; other days unaffected	as Delta
Node state lost	AWS Backup of EFS; caches rebuild from the lake	**BUILT** (scripts)

The Iceberg-specific DR rule: absolute paths make a copy a different table until `relocate` maps it; with a REST catalog, the catalog's metadata location must be repointed in the recovery Region too.

17

Limits and what comes next

What is not yet proven, known limits, the decisions to take, and where to read more

Limits and what comes next

- **Not yet run in AWS.** No Iceberg table written to a real S3 bucket; Glue's REST endpoint never called; S3 Tables untried
- **Risk workloads are Delta only:** VaR and exposure vectors, official results (and no publishing to `s3://` in either format)
- **No AWS scripts for Iceberg:** load, backfill, maintain, lifecycle prefixes, alarms and report are Delta-shaped
- **Maintenance gaps:** no orphan-file removal, no Admin → Lakes, no 7-year floor for Iceberg risk tables
- **Spec gaps, refused by name:** format v3 (deletion vectors, new types), encrypted tables, ORC and Avro data files, branches and tags
- **Native engine reaches local disk and S3 only;** Azure, GCS, HDFS need `engine: library` (Hadoop)
- **Cluster:** Iceberg reports no table identity to Drishti*net; VaR sharding across a cluster of Drishti servers is designed, not built
- **Doc fixes to make:** ICEBERG_CONNECTOR.md section 4 (path tables on S3 'need a single writer': true only of the library engine); VAR_GUIDE.md 29.3 (Iceberg 'the same list<double> column': not for VaR's reads)

Decisions and actions for the product owner

#	Decision or action	Recommendation
1	Which format for the **risk domain** (vectors, official results)	Delta now; Iceberg after the plugin's `vectors()` and an Iceberg publisher exist
2	Which format for **trades and reference data**	either, per source system; Iceberg where the bank's pipelines already write it
3	The **catalog** on AWS for Iceberg	path-based first; Glue's REST endpoint when Athena or Spark must share the tables; S3 Tables only after a proof
4	First real run in a **development AWS account**	Delta steps 0–11 and an Iceberg load and read of one table, with `check-conditional-writes.sh`
5	Close the Iceberg gaps (section 12's list of six)	as one wave: plugin methods, AWS scripts, maintenance, identity, publishing
6	Seven-year history on Iceberg	keep full-run snapshots, expire intraday ones; measure metadata at 2,000+ snapshots first

Related documents

Document	What it holds
docs/architecture/DRISHTI_ICEBERG.md	the native engine's design: protocol coverage, catalogs, commits, two connectors
docs/connectors/ICEBERG_CONNECTOR.md	the `iceberg` connector: layout, catalogs, loader, reads, maintenance, engines, settings
docs/connectors/ICEBERG_EXTERNAL_CONNECTOR.md	`iceberg-external`: tables Drishti does not write
docs/architecture/AWS_DELTA_LAKE.md	the Delta lake on AWS: scripts, layout, loading, maintenance, cost, runbooks
docs/architecture/AWS_DEPLOYMENT.md	ECS, EFS and the identical-path rule; buckets; secrets; DR
docs/connectors/DELTA_CONNECTOR.md	how Drishti reads Delta; engines native and Hadoop
docs/admin/LAKE_MAINTENANCE.md	`lake maintain` and Admin → Lakes (Delta); the 7-year risk floor
docs/guides/VAR_GUIDE.md	VaR inputs (15), official results (19), sizing (27), sharding (28), vectors in the lake (29)
docs/connectors/DEMO_DATA.md	mixed Delta and Iceberg storage in the demo
docs/architecture/DRISHTI_NET.md, docs/admin/NET.md	several servers: replicas, shards, routing